

Research Article

Pothole Detection and Assessment on Highways Using Enhanced YOLO Algorithm With Attention Mechanisms

Rufus Rubin ¹, Chinnu Jacob ¹, Sumod Sundar ¹, Gabriel Stoian ²,
 Daniela Danciulescu ² and Jude Hemanth ³

¹Centre for Artificial Intelligence, TKM College of Engineering, Kollam, India

²Department of Computer Science, University of Craiova, Craiova, Romania

³Department of ECE, Karunya Institute of Technology and Sciences, Coimbatore, India

Correspondence should be addressed to Jude Hemanth; judehemanth@karunya.edu

Received 2 October 2024; Accepted 26 February 2025

Academic Editor: Gang Mei

Copyright © 2025 Rufus Rubin et al. Advances in Civil Engineering published by John Wiley & Sons Ltd. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Economic and social prosperity heavily relies on well-maintained highways. However, road maintenance faces challenges due to limited funding and resources, with potholes posing significant safety risks. This work introduces a pothole detector designed to detect and estimate pothole areas for timely maintenance. It enhances detection by modifying the YOLO algorithm, using the Xception backbone, and integrating attention mechanisms to improve the prediction of small or clustered objects. Xception's depthwise separable convolutions enhance feature extraction, outperforming the standard YOLO algorithm in detecting small, irregular potholes and preventing overfitting. The improved YOLO model, along with spatial and channel attention mechanisms, focuses on relevant regions and refines important features specific to pothole areas. Accurate area estimation is achieved through computer vision and traditional segmentation processes. A custom dataset, including the MakeML pothole dataset, a Kaggle dataset, and real-time footage of Kerala roadways, is used for training and validation. Performance evaluation with mean average precision (mAP) and average precision (AP) metrics shows the pothole detector's superiority, effectively identifying potholes under various conditions and ensuring safe road infrastructure.

Keywords: area estimation; convolution block attention module (CBAM); detection; pothole; Xception

1. Introduction

Roads have a significant impact on the most prevalent mode of transportation, and our reliance on them has increased substantially. Consequently, the physical characteristics of roads have undergone significant changes. To prevent accidents and transportation disruptions, it is crucial to maintain roads effectively and appropriately. In India, road transportation accidents are influenced by various factors such as potholes, uneven manholes, low skid resistance, aquaplaning during rainfall, and reckless driving. Potholes, in particular, contribute to a majority of accidents, which is one of the reasons why India has one of the highest accident rates worldwide. As per the Ministry of Road Transport and

Highways, in 2020, there were a total of 366,138 recorded road accidents, resulting in the loss of 131,714 lives and injuring 348,279 individuals [1]. Alarming, the age group most impacted by road accidents is 18–45 years old, comprising about 70% of all accidental fatalities. The economic implications are equally significant. A report by the Transport Research Wing estimates that road accidents cost India ~3% of its GDP annually, with vehicle damage and traffic delays caused by poor road conditions, including potholes, contributing substantially [2].

Potholes are significant structural defects that occur on roads, caused by water infiltrating the underlying soil and weakening the road's structure through repeated expansion and contraction. The continuous weight of passing vehicles

further contributes to the fracturing and dislodging of the deteriorating road surface. While not typically classified as structural failures, potholes pose a serious threat to both vehicles and human lives [3]. While not traditionally classified as structural failures, potholes pose a serious threat to both vehicles and human lives. For instance, studies have shown that potholes are directly responsible for significant vehicle damage, including tire blowouts, suspension system failures, and misalignment, which collectively cost motorists billions of dollars annually worldwide [4]. Additionally, potholes contribute to traffic congestion by slowing vehicle speeds and increasing fuel consumption, further compounding economic losses [5].

Despite their critical impact, the current approach to pothole management, which relies on manual road patrols and inspection teams, is often time-consuming and labor-intensive. This delay in identifying and repairing potholes not only increases the risk of accidents but also escalates maintenance costs. To address this issue, there is a growing need for automated systems capable of detecting and reporting potholes in real-time. A proposed solution involves equipping vehicles with sensors and cameras to autonomously detect potholes during transit. The detected pothole locations can be stored on a cloud server and overlaid on digital road maps. This information can then be used to dispatch maintenance vehicles with optimized routes based on pothole densities and severities, ensuring efficient and timely repairs [6].

This work proposes a computer-assisted system for automatic pothole detection using advanced deep-learning techniques integrated with real-time sensor data. The methodology involves leveraging attention mechanisms to enhance the detection of small and clustered potholes, ensuring high accuracy and reliability. The remainder of the work is structured as follows: Section 2 provides a detailed review of related works, highlighting existing challenges and gaps in pothole detection systems. Section 3 elaborates on the proposed methodology, including the architecture, dataset preparation, and training strategies. Section 4 presents experimental results, comparing the proposed approach with state-of-the-art methods. Section 5 discusses the implications of the findings, and Section 6 concludes the work with recommendations for future work.

By developing reliable and standardized techniques for automatic pothole detection, this research aims to mitigate the challenges posed by potholes, enhancing road safety and reducing economic losses.

2. Related Works

Deep-learning techniques have significantly advanced detection systems, particularly in the transportation sector, by leveraging their ability to extract and process features from large datasets. These techniques have also been widely applied beyond transportation, demonstrating their versatility in domains such as medical imaging, object detection, and urban planning [1, 2]. The adaptability of deep-learning algorithms, combined with customization for specific use cases, has yielded robust systems that consistently outperform traditional methods in various scenarios. For instance,

Jacob and Menon [7] and Sundar and Sumathy [8, 9] demonstrated the importance of tailoring neural networks to extract domain-specific features effectively, highlighting the potential for enhancing detection accuracy across multiple fields.

In transportation, the use of deep learning models has expanded to include external and internal feature utilization for automated systems [10]. For example, LSTM models are employed in intelligent transportation systems to predict optimal routes with minimal travel time [11], while multi-layered deep-learning approaches have shown promise for forecasting traffic congestion [12].

Detection techniques in transportation numerous approaches have been proposed for road condition monitoring and pothole detection. Ochoa-Ruiz et al. [5] employed RetinaNet to detect road damage using a custom dataset of 18,435 images captured by smartphones mounted on vehicles. While their approach achieved notable success, its effectiveness is constrained by dataset-specific characteristics, limiting its generalizability to diverse road surfaces and geographical regions.

Gupta et al. [2] modified single-shot multibox detector (SSD [13]) models by integrating ResNet-34 and ResNet-50 as backbones. Their approach included eliminating the fully connected ResNet-34 layers and introducing additional convolutional layers. The modified ResNet-50, equipped with a feature pyramid network (FPN), delivered superior results compared to the ResNet-34 variant. However, relying solely on thermal images presented challenges, particularly in poor lighting conditions or when the thermal contrast between potholes and the surrounding road surface was insufficient.

Luo [4] developed a deep-learning pothole detector (DLPD) comprising a region proposal network (RPN), a convolutional neural network (CNN), and layers akin to Faster R-CNN. Ahmed [6] enhanced Faster R-CNN by modifying the VGG-16 architecture (MVGG16) to reduce computational time and improve training efficiency. MVGG16 introduced dilated convolutions to increase the receptive field without increasing computational overhead.

Asad et al. [14] explored YOLO-V2, YOLO-V3, and YOLO-V5 for pothole detection and deployed these models on edge devices, such as AI Kit (OAK-D) integrated with Raspberry Pi. Their system achieved a high mean average precision (mAP) of 95%. However, it struggled with misclassification and nondetection at longer distances. Similarly, P. B. and K.C. [15] adapted the YOLO-X model, achieving an average precision (AP) of 85.6%.

From these studies, it is evident that many methods rely on complex architectures, such as Faster R-CNN or FPN-enhanced backbones, which are computationally intensive and may lack explainability. This motivates the development of simpler, yet effective, models that can generalize well across varying lighting conditions and road types.

Recent advancements have highlighted the role of generative adversarial networks (GANs) in enhancing image quality for detection tasks. GANs have been used to generate synthetic training data, improve image resolution, and enhance features for detecting small objects. For instance, Li et al. [16] utilized GANs to enhance low-resolution images, enabling

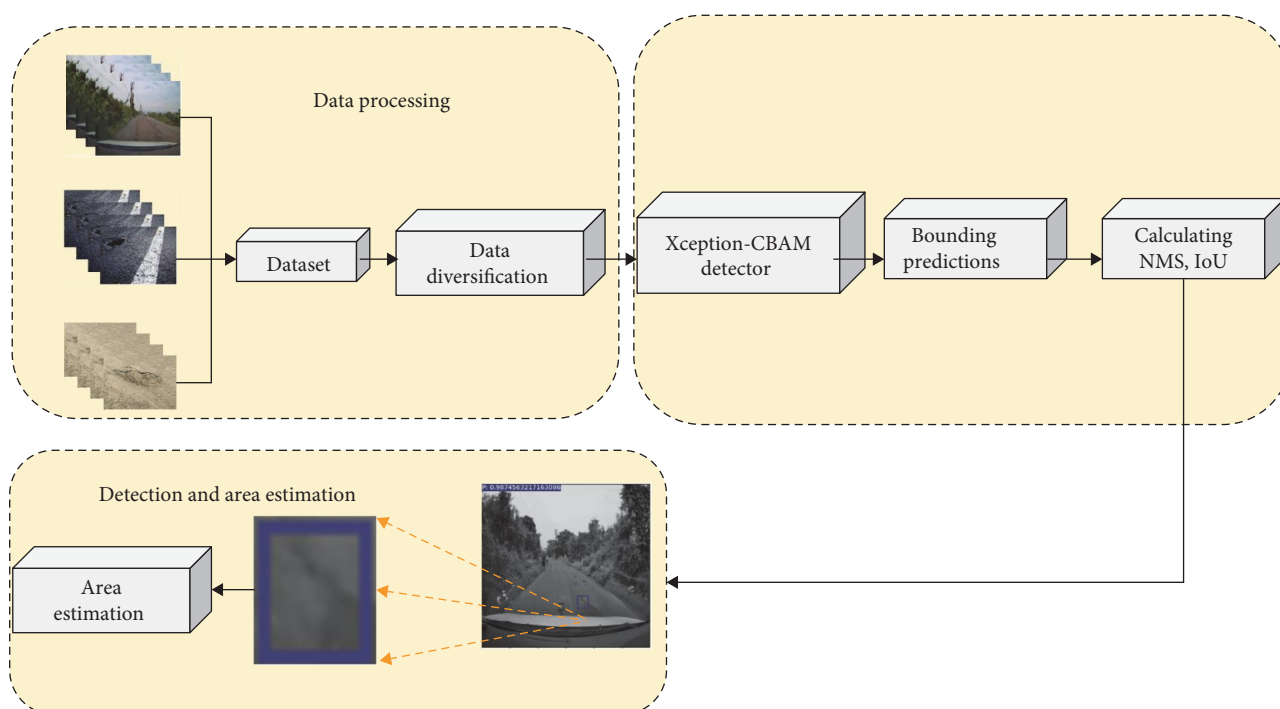


FIGURE 1: Proposed methodology.

better feature extraction for small object detection. Similarly, Zhong et al. [17] applied CycleGAN to augment datasets by generating diverse road surface textures, improving the robustness of pothole detection models.

Incorporating GANs into pothole detection can address challenges such as poor lighting conditions and low-resolution data. By generating enhanced images with better feature contrast, GANs can facilitate the detection of smaller or obscured potholes. For example, GAN-based techniques like super-resolution GAN (SRGAN) and DeblurGAN can be employed to preprocess images captured under challenging conditions, improving the overall accuracy and robustness of the detection system [18, 19].

Building on these advancements, our proposed model aims to address the limitations of existing techniques by focusing on simplicity, explainability, and adaptability. By avoiding overly complex architectures, our approach ensures computational efficiency while maintaining high detection accuracy. Furthermore, we explore integrating GAN-based preprocessing methods to enhance image quality, ensuring effective operation in diverse environmental and road surface conditions. This approach seeks to bridge the gap between traditional and modern techniques, providing a holistic solution for pothole detection in transportation systems.

2.1. Contributions of the Work. The work presents a model that combines both computer vision processing and deep learning techniques to detect and estimate the size of potholes. The significant contributions of this work are outlined below:

- i. A pothole detector is proposed by leveraging the YOLO algorithm and exploring different backbone architectures.
- ii. To enhance the accuracy of detection, an attention mechanism is incorporated in the pothole detector.
- iii. An augmentation method using data normalization is being utilized to achieve mixed diversification.
- iv. A computer vision technique is being applied to accurately estimate the area.
- v. The Xception convolutional block attention module (CBAM)-based pothole detector demonstrates superior performance compared to other models in detecting potholes and ensuring road safety.

3. Materials and Methods

The proposed pothole detector utilizes a CNN model to identify and locate single potholes. Initially, a mixed dataset of images is provided for training the CNN model. The pothole detector employs Xception-CBAM layers, followed by average pooling, flatten, and dense layers, and incorporates an attention mechanism to enhance performance. Once a pothole is detected in the given images, it is cropped based on bounding boxes and further analyzed using computer vision techniques to estimate the area of the potholes. The system's workflow is illustrated in Figure 1.

3.1. Dataset Used. In this work, we utilized the MakeML pothole dataset, a publicly available dataset, for our analysis. The dataset consists of pothole images collected from

TABLE 1: Dataset information.

Dataset	Number of images
MakeML pothole	665
Custom video dataset 1 (17:00 min)	432
Custom video dataset 2 (1:00 min)	252
Kaggle dataset	500
Total	1849

multiple sources, including (MakeML [20] and Roboflow [21]). The dataset consists of 665 images. The dataset includes bounding box annotations in PASCAL visual object challenge (VOC) XML format, which were used to create detection models. This dataset can serve as a proof of concept (POC)/proof of value (POV) for road maintenance purposes. The dataset contains around 2139 pothole images, and each image is of size 400×400 . The annotations file is in the format of Pascal VOC XML. Pascal VOC XML is the annotation format that was initially developed for the VOC and has since developed into a standard interchange format for object detection labels.

The second dataset is created by our own source. The dataset is taken by a camera mounted on the car and contains two videos: the first video is in 17:00 min time duration and the second video is in 1:00 min time duration. Both videos are taken from different illumination scenarios. Then, converted both videos to frames. The frames taken from the first video are 432 images, and from the second video, the frames taken are 252 images. The image size of the both video frames is (1920×1080) . The images were manually annotated using an online platform called Roboflow. Roboflow is a platform that offers a collection of tools and services for the administration and preparation of computer vision datasets. It provides capabilities such as annotating, augmenting, and transforming data, as well as training and deploying models. Users are able to easily manage and organize their data with the help of Roboflow, as well as preprocess it and use it to train and test computer vision models. The platform offers support for a wide variety of data formats and has the capability to integrate with widely used deep learning frameworks. As a result, it is a versatile and flexible solution for computer vision tasks. The third dataset is the Kaggle pothole dataset [22]. It contains 500 images, and the size of the images is (690×920) , and the bounding box annotations are provided in the above format. The distribution of data across three datasets is presented in Table 1. The sample images of each dataset are shown in Figure 2.

After using the enhancing generalization method, the data are increased to 3698, then split into 70% training, 20% validation, and 10% testing.

3.2. Enhancing Generalization With Mixed Data Representation Using Normal Distribution (Data Diversification). Enhancing generalization through mixed data representation involves combining normalized images with their original, unnormalized versions during training, fostering diversity in the training dataset, and improving the model's adaptability to real-world variations. This approach leverages data normalization, a

well-established preprocessing technique in computer vision, to scale pixel values within red, green, blue (RGB) channel images to have a mean of zero and a standard deviation of one. This process effectively addresses lighting and contrast disparities, enabling the model to learn more robust features [1].

In this study, data normalization was performed by calculating the mean and standard deviation along the channel axis, establishing a transformation pipeline, and applying normalization to tensor images. This preprocessing step standardizes input data, promoting a more efficient training process and leading to improved model accuracy. Figure 3 visually illustrates the transformation of pixel value distributions due to normalization, highlighting the shift from a wide range of values (0–255) for each channel before normalization to a consistent distribution with a mean of zero and a standard deviation of one after normalization.

To address potential dataset biases, we ensured diversity in the dataset by including road images from various geographical regions, lighting conditions, and road types. However, we acknowledge that inherent biases might still exist due to limited representation of certain regions or specific road conditions. Future work will involve expanding the dataset by incorporating data from unstructured, real-world sources, such as crowdsourced imagery and publicly available road datasets, to further improve model generalization.

The concept of using mixed data representations to enhance model generalization is supported by prior studies, such as Xie et al. [23], which demonstrated the effectiveness of combining original and augmented data for robust feature learning. Similarly, Krizhevsky et al. [24] highlighted the impact of normalization on training efficiency and model performance in their seminal work on AlexNet. By combining normalized and unnormalized images during training, the model is exposed to diverse data representations, enhancing its adaptability to unseen scenarios during testing.

3.3. Proposed Architecture. The architecture of a pothole detector is shown in Figure 4. An Xception network serves as the backbone for the pothole detector, with the addition of an integrated attention mechanism. This incorporation of attention aims to improve the feature representation abilities of the CNNs by emphasizing important features and reducing the impact of irrelevant ones.

In the proposed methodology, we employ the Xception [25] deep neural network architecture, consisting of three distinctive flows: the entry flow, middle flow, and exit flow. The entry flow incorporates a classical convolutional block along with three residual separable convolution blocks. The classical convolution block facilitates the extraction of low-level features from the input image. The resulting output is then forwarded to the residual separable convolution block, which integrates depth-wise separable convolution with a linear residual connection. This block effectively reduces parameters and computational requirements while preserving accuracy.

Moving to the middle flow, it comprises eight residual separable convolution blocks, aiming to extract complex features from the input data by stacking multiple residual



FIGURE 2: Pothole images: (a) MakeML pothole, (b) custom dataset, and (c) Kaggle dataset.

separable convolution blocks. Finally, the exit flow comprises one residual separable convolution block and one separable convolution block, extracting high-level features from the input tensor. Notably, in the Xception backbone, structural changes have been introduced by integrating the CBAM at the end to emphasize important features. The key structural change involves the removal of certain exit flow layers and their replacement with the CBAM mechanism, strategically focusing on essential features.

The CBAM mechanism, as proposed by Woo et al. [26], plays an important role in enhancing the representational performance of CNNs. This enhancement is achieved by emphasizing informative features and suppressing irrelevant ones. The CBAM module comprises two integral components: the channel attention module (CAM; CBAM) and the spatial attention module (SAM). Both modules operate on the feature maps generated by a convolutional layer in the CNN, generating weighted feature maps that are more informative and relevant while concurrently suppressing irrelevant features. The CBAM processes the feature map, producing a one-dimensional channel attention map denoted as $M_c \in R^{C \times 1 \times 1}$ and a two-dimensional spatial attention map denoted as $M_s \in R^{1 \times H \times W}$. These attention maps collectively contribute to the refined and focused feature extraction process, as explained in detail in the architecture details section.

The CAM operates on the channel dimension of the feature maps and learns to assign different weights to channels based on their relative importance. This is accomplished by employing a global average pooling (GAP) operation on the feature maps, followed by two fully connected layers that produce a set of channelwise attention weights utilized to reweight the feature maps. First, to determine the values of average-pooled features F_c^{avg} and then max-pooled features, respectively F_c^{max} . These values are then feed to the shared network to produce our channel Attention map denoted as $M_c \in R^{C \times 1 \times 1}$. The shared network is comprised of a multi-layer perceptron (MLP) with one hidden layer. Channel attention is determined by using Equations (5) and (6), which are given below [25]:

$$M_c(F) = \text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{Maxpool}(F)), \quad (1)$$

$$M_c(F) = W_0(W_0(F_c^{avg})) + W_1(W_0(F_c^{max})), \quad (2)$$

where $M_c(F)$ represents the output of the SAM for the input feature map F on the channel dimension. The F_c^{avg} and F_c^{max} represent the average-pooled and max-pooled feature maps, respectively, along the channel axis. W_0 and W_1 are weight values used to scale the respective feature maps and spatial attention maps. The operations $W_0(F_c^{avg})$ and $W_0(F_c^{max})$ indicate element-wise multiplication of the feature maps with their corresponding weights, and the results are then multiplied by the spatial attention map.

The SAM operates on spatial dimensions, selectively focusing on informative locations via max pooling and average pooling along the channel axis. The max-pooled feature is denoted by $F^{max} \in R^{1 \times H \times W}$ and the average pooled features is denoted by $F^{avg} \in R^{1 \times H \times W}$. These two features are combined to apply a convolution process to generate a spatial attention map, which is denoted as $M_s(F) \in R^{H \times W}$. The SAM allows the model to focus on important spatial locations by combining information from both average-pooled and max-pooled feature maps and weighting them with learned parameters W_0 and W_1 before generating the spatial attention map $M_s(F)$.

$$M_s(F) = (f_{7 \times 7}[\text{AvgPool}(F); \text{MaxPool}(F)]), \quad (3)$$

$$M_s(F) = (f_{7 \times 7}([F^{avg}, F^{max}])). \quad (4)$$

In Equations (3) and (4), $f_{7 \times 7}$ represents a convolution operation with a 7×7 filter, while the sigmoid function is applied to generate the spatial attention map $M_s(F)$, ensuring the output values remain within a normalized range.

The final weighted feature maps are produced by combining the CAM and SAM by multiplying the channel-wise attention weights and the spatial attention weights element-wise. The final weighted feature maps are then processed by a convolutional layer to generate the final output.

3.3.1. Architecture Details. In the Xception model, the input tensor denoted as “ x ” with dimensions (H, W, C) undergoes a series of block functions as it progresses through the entry

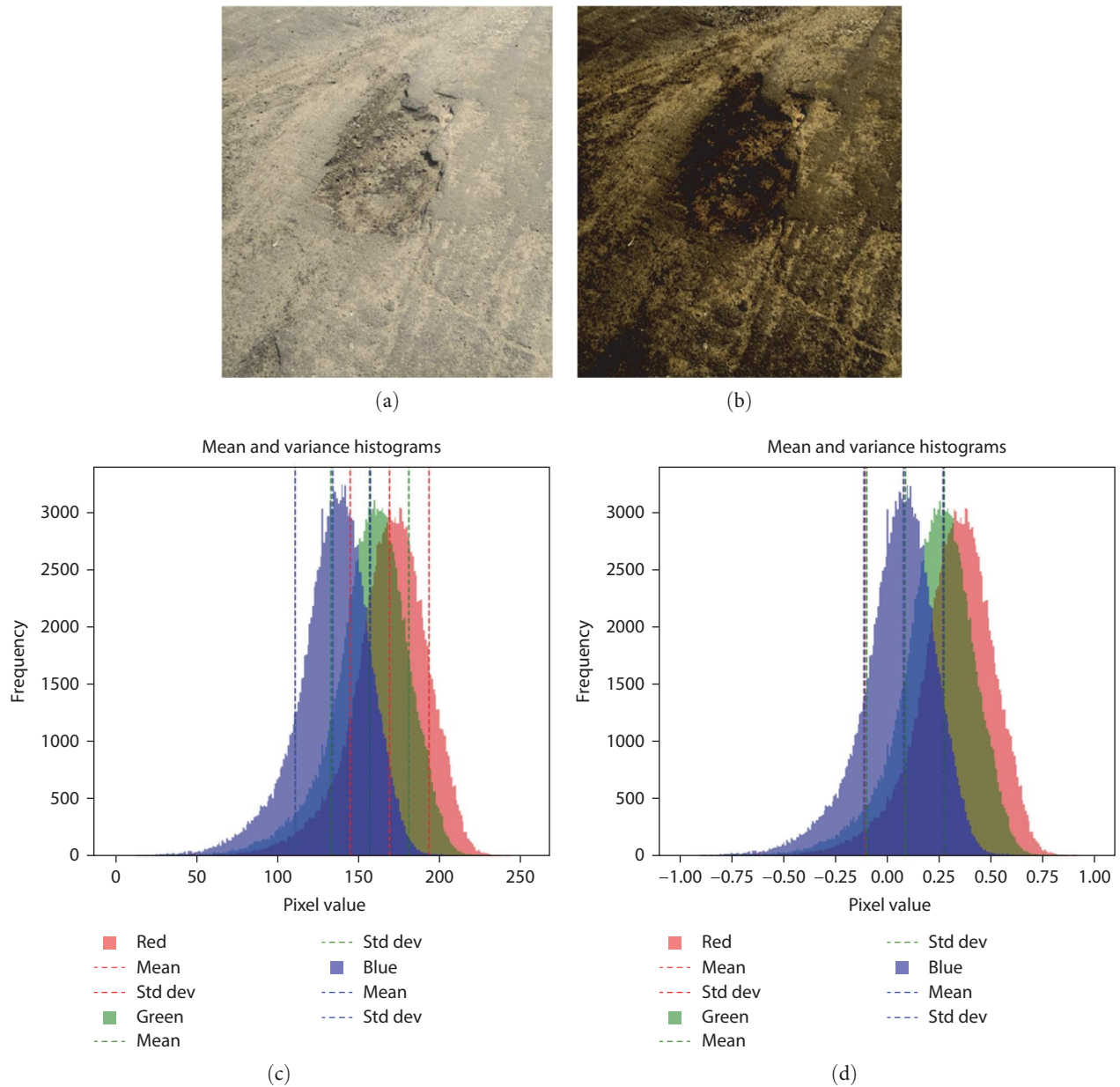


FIGURE 3: Normalize analysis of pothole images with the normal distribution of image values: (a) pothole images, (b) pothole image after normalization, (c) mean and variance of pothole image, and (d) mean and variance of pothole image after normalization.

Data: Tensor image

Output: Normalized image

Read the input image

- Calculate the mean and standard deviation of the tensor image along the channel axis, which has index 1. This will result in two tensors, mean and standard, each with channel elements.
- Subtract the mean tensor from the tensor image.
- Divide the resulting tensor by the standard deviation tensor.
- Return the normalized tensor image. Repeat the steps for images also.

ALGORITHM 1: Data Normalization Algorithm

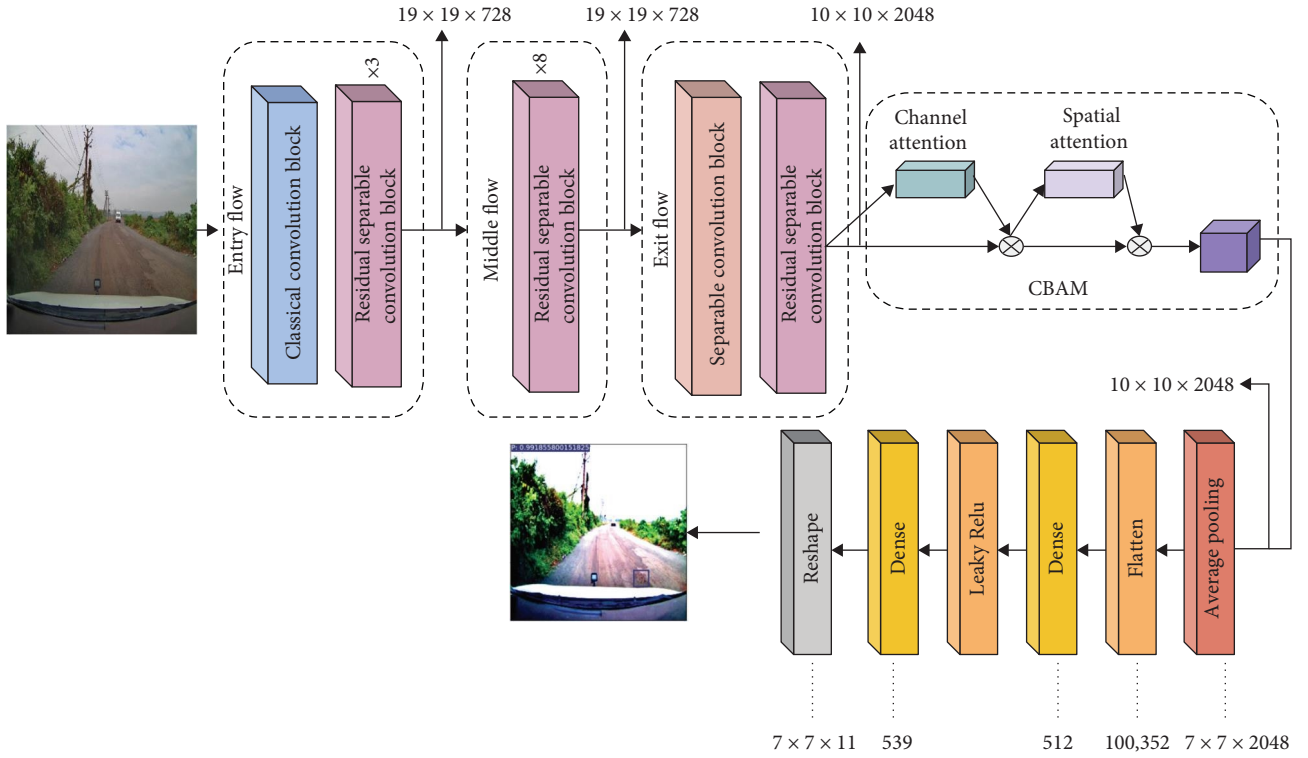


FIGURE 4: Proposed architecture.

flow, middle flow, and exit flow for processing. In this context, F_{XEF} , F_{XMF} , $F_{XE \times F}$ represent the output tensors after applying these functions, respectively. Additionally, a CBAM is applied to the output of the Xception exit flow.

$$F_{XEF} = F(X)_{XEF}, \quad (5)$$

$$F_{XMF} = F(X_{XEF})_{XMF}, \quad (6)$$

$$F_{XEXF} = F(F(X_{XMF}))_{XEXE}. \quad (7)$$

The CBAM attention mechanism consists of two core operations: channel attention and spatial attention. Channel attention involves GAP and global max pooling (GMP), performed along spatial dimensions on the output tensor $F(X)$. These operations create two distinct vectors. Channel attention is calculated by combining these vectors through addition and applying a sigmoid activation to obtain the CAM.

$$f_{GAP} = f(F(X))_{GAP}, \quad (8)$$

$$f_{GMP} = f(F(X))_{GMP}. \quad (9)$$

In spatial attention, a SAM performs max pooling along the spatial dimension of $F(X)$, followed by a sigmoid activation. The channel attention map and spatial attention map are combined element-wise through multiplication to create the overall attention map, which is used to reweight the original input tensor $F(X)$. The reweighted tensor, denoted

as "OutputTensor," is obtained by scaling $F(X)$ using the overall attention map, where "K" is the kernel size and "S" is the stride.

$$\text{favpool}_{(K,S)} = f(F(X))\text{avgpool}_{(K,S)}, \quad (10)$$

$$\text{fmaxpool}_{(K,S)} = f(F(X))\text{maxpool}_{(K,S)}, \quad (11)$$

$$f_{CAV} = \text{favpool}_{(K,S)} + \text{fmaxpool}_{(K,S)}. \quad (12)$$

Apply sigmoid activation $f_{CAM} = \sigma(f_{CAV})$, where the CAM is the channel attention mechanism, and CAV is the channel attention vector. In spatial attention, SAM performs max pooling along the spatial dimension of $F(X)$ and applies sigmoid activation.

$$f_{SAM} = f_{GAP} = f(F(X)SAM), \quad (13)$$

$$f_{SAM} = \sigma(f_{SAM}). \quad (14)$$

Combine Channel and Spatial Attention: combine the channel attention map and spatial attention map to obtain the overall attention map, which is used to reweight the original input tensor $F(X)$. Combine f_{CAM} and f_{SAM} element-wise using multiplication and scale the original tensor $F(X)$ by the overall attention map.

$$\text{Overall attention} = f_{CAM} \times f_{SAM}, \quad (15)$$

$$\text{Output tensor} = F(X) \times \text{overall attention.} \quad (16)$$

Subsequently, the output tensor is subjected to adaptive average pooling, with an output size of 7. After further processing through flatten and dense layers, the resulting output is reshaped to a $(7 \times 7 \times 11)$ structure, where 7×7 represents the grid size, and 11 signifies the presence of two bounding boxes. Each bounding box contains five elements: confidence score, center coordinates (x and y axes), height, width, and a class label of 1. These steps collectively describe the intricate operations within the Xception model, incorporating CBAM for enhanced attention and processing of the input tensor.

The incorporation of attention mechanisms, such as CBAM, into CNNs has been shown to significantly improve feature representation abilities. CBAM allows the network to focus on the most relevant features by adaptively adjusting the importance of different channels and spatial regions, leading to improved performance in tasks such as object detection and classification.

Several studies have demonstrated the effectiveness of attention mechanisms in enhancing CNNs. For instance, Woo et al. [26] introduced the CBAM module and showed that combining both channel and spatial attention improves the model's performance on image classification tasks. Similarly, Zhang et al. [27] applied attention mechanisms to semantic segmentation, demonstrating their ability to enhance feature extraction and improve accuracy. These

findings highlight the benefits of incorporating attention mechanisms like CBAM in improving the model's ability to focus on relevant features, which is crucial for tasks involving complex visual data, such as pothole detection in road images.

3.3.2. Loss Measures. In the pothole detector neural network, a fundamental component called the YOLOV1 loss function [28] is employed to quantify the disparity between predicted and actual outputs. This loss function comprises three crucial elements: localization loss, confidence loss, and classification loss. The localization loss measures the difference between predicted and actual bounding box coordinates within the grid cell containing the object's center by summing squared errors. The confidence loss evaluates the accuracy of object presence predictions within grid cells using binary cross-entropy between predicted and ground truth objectness scores. These objectness scores represent the likelihood of objects being present within their respective bounding boxes. Additionally, the classification loss quantifies errors in predicted class probabilities for each object by summing squared differences with true class probabilities. Together, these three components form the YOLO-V1 loss function, and their relative contributions are controlled by hyperparameters (λ_{coord} , λ_{obj} , and λ_{noobj}). Fine-tuning these hyperparameters through a grid search process is essential for optimizing model performance.

$$\begin{aligned} \text{Loss} = & \lambda_{\text{coord}} \cdot \text{localization loss} + \lambda_{\text{obj}} \cdot \text{confidence loss} \\ & + \lambda_{\text{noobj}} \cdot \text{no-object confidence loss} + \lambda_{\text{class}} \cdot \text{classification loss} \end{aligned} \quad (17)$$

where

- localization loss is the sum over bounding boxes, and it can be 0 or 1 (if the box contains an object, otherwise it will be 0) and is multiplied by the sum over box coordinates (it is the squared error between the predicted and true box coordinates normalized by the image width and height);
- confidence loss is the sum over grid cells (1 if the cell contains an object, otherwise it will be 0) and is multiplied by the squared error between predicted and true objectness scores multiplied by the objectness indicator function;
- no-object confidence loss is the sum over grid cells (0 if the cell contains an object, 1 otherwise) and is multiplied by the squared error between predicted and true objectness scores multiplied by the objectness indicator function;
- classification loss is the sum over bounding boxes (1 if the box contains an object, 0 otherwise) and is multiplied by the sum over classes (squared error between predicted and true class probabilities multiplied by class indicator function).

3.3.3. Optimizer. The core principle of optimization is to minimize the loss function by adjusting the model parameters during training. The goal is to find parameter values that minimize the cost function, achieved through iterative computation of the gradient of the loss function with respect to model parameters and subsequent updates that reduce the cost. In this study, stochastic gradient descent (SGD) was employed as the optimizer. SGD calculates gradients using random data subsets known as mini-batches, resulting in frequent parameter updates that can introduce fluctuations in the loss function. The learning rate also called the step size, plays a crucial role in determining the magnitude of each parameter update and significantly influences the convergence towards the minimum. The iterations of SGD can be described as involving iterates denoted as W_k ; step sizes represented as α_k (learning rates), and stochastic gradients computed at w_k denoted as $\nabla f(w_k)$ [29, 30].

$$w_k = w_k - 1 - \alpha_k - 1 \nabla f(w_k - 1). \quad (18)$$

SGD performs parameter updates for training data. For example, $x^{(i)}$ is a training sample, and $y^{(i)}$ is the label, then we can write as an equation:

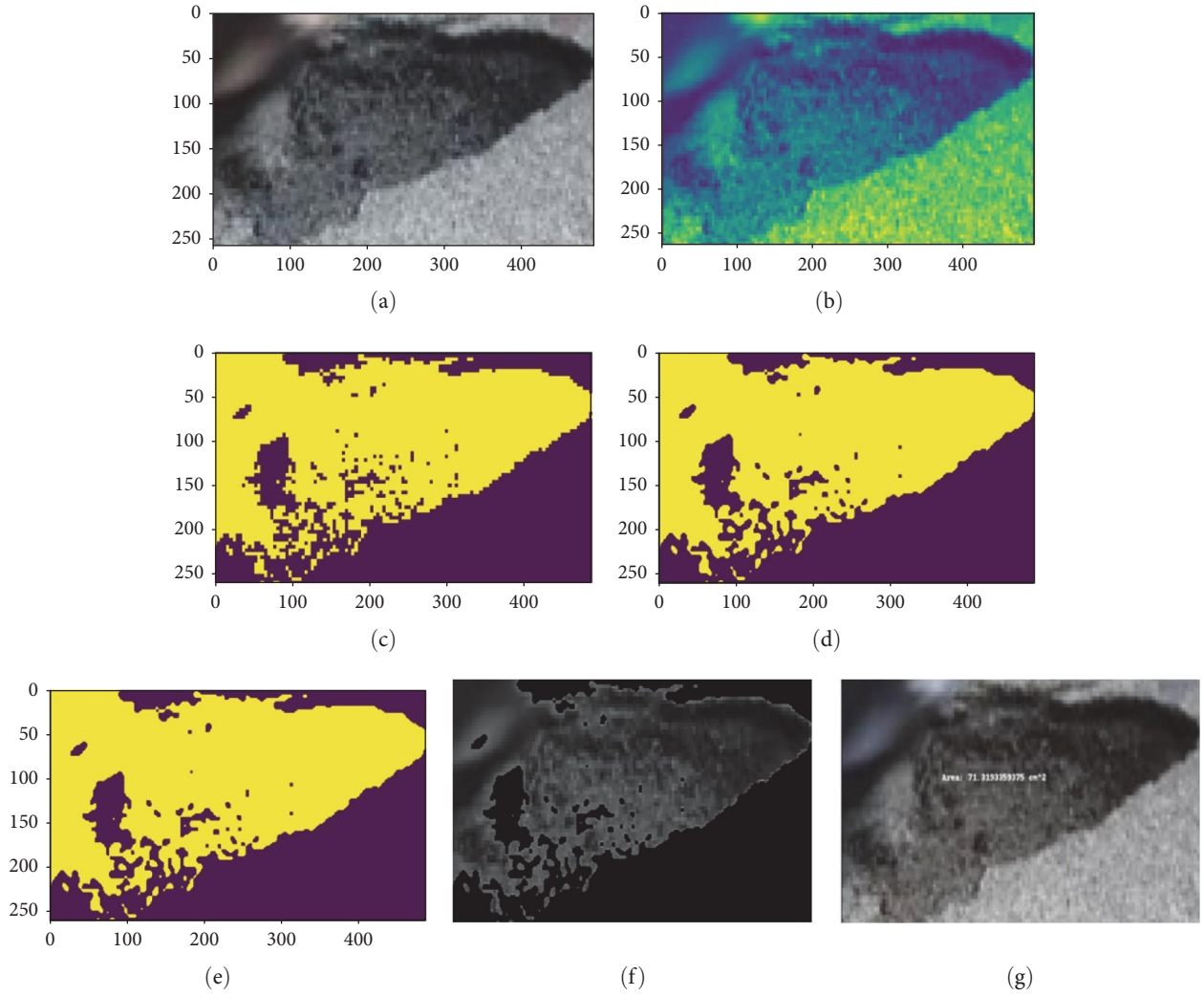


FIGURE 5: (a) Pothole image, (b) grayscale, (c) inverted binary threshold image, (d) median blur image, (e) morphology close image, (f) overlay image, and (g) area estimated image.

$$\theta = \theta - \eta \cdot \nabla J(\theta; x(i); y(i)), \quad (19)$$

where $J(\theta)$ is the objective function parameterized by a model parameter θ by updating the parameters in the opposite direction of the gradient of the objective function $\nabla_{\theta} J(\theta)$ with respect to the parameters. η is the learning rate that determines the step sizes to reach the minimum. The advantage of this SGD is the frequent update approach; it allows the algorithm to learn quickly from the data, especially in high-dimensional spaces, as it takes only one example at a time to perform a weight update. Additionally, this approach can also help to avoid local minima, as the stochasticity in the gradient estimates can allow the algorithm to jump out of small valleys and find more optimal solutions.

3.4. Area Estimation. Potholes are a common issue on roads, highways, and streets that pose a considerable risk to drivers, pedestrians, and cyclists. They can also cause significant damage to vehicles. Therefore, accurately estimating the area of a pothole is crucial for determining the materials needed to repair it. Several techniques can be used to

estimate the area of a pothole, including grayscale analysis, inverted binary thresholding, morphology processing, and median blur. Figure 5 shows the result of area estimation using computer vision techniques. By estimating the area of the pothole, road maintenance teams can accurately determine the amount of repair materials required, helping to minimize waste and ensure that repairs are completed efficiently. This can ultimately help to reduce the risks posed by potholes and improve the safety of roads, highways, and streets for all users. The various steps in area estimation are shown in Algorithm 2:

- **Image Acquisition and Preprocessing:** A dataset of images containing potholes is received. The identified potholes are isolated by cropping them based on their respective bounding boxes. Subsequently, the cropped pothole images are converted into grayscale for further analysis.
- **Inverted Binary Thresholding and Morphological Close Operation:** Inverted binary thresholding is applied to the grayscale image, resulting in a binary image with

Data: Crop the image detected by the pothole detector.

Output: Estimated the area of pothole.

- Input the cropped pothole image according to bounding box.
- Convert the cropped image into grayscale.
- Apply the inverted binary threshold on the grayscale image
- Take the median blur to remove the noise.
- Apply the morphology close technique to the median blur image.
- Take the absolute difference between the grayscale and morphological closing image to extract the edges and contours.
- Apply the binary threshold technique to make it easier to segment the area.
- Calculate the pixels and convert it into centimeters. Repeat the steps for pothole images.

ALGORITHM 2: Area Estimation Algorithm

reversed colors. The dark pixels in the grayscale image become white in the binary image, and bright pixels become black. This process emphasizes the areas of interest, usually dark regions in the grayscale image. Then, a morphological close operation is performed to refine edges, eliminate small holes or gaps, and smooth object edges.

- **Median Blur for Noise Reduction:** To enhance the effectiveness of the morphological close operation, a median blur operation is applied to the grayscale image first. This technique reduces random noise, such as salt-and-pepper noise, by replacing each pixel value with the median value of its neighboring pixels within a specified kernel or window size. Smoothing the edges of objects ensures that the close operation is applied accurately, excluding noise-induced gaps or holes.
- **Absolute Difference and Binary Thresholding:** The absolute difference between the grayscale image and the output of the morphological closing image is computed. This step emphasizes the differences between the two images, particularly at the edges of the objects, and helps to extract edges and contours from the image. Afterward, binary thresholding is applied to segment the pothole area, making it easier to identify and isolate the pothole region.
- **Size Calculation:** After segmenting and isolating the area of interest through image processing, the number of white pixels within that segment is counted. To convert this pixel count into a measurable physical unit, like square centimeters, a calibration reference with a known physical dimension within the image is required. The area of each pixel in square centimeters is determined based on the image's resolution in both the x and y directions. Finally, the size of the segment in square centimeters is obtained by dividing the number of white pixels by the area of each pixel. This allows for the translation of the digital representation of the area into a physical measurement.

To calculate the size of the segment area in square centimeters, you simply divide the number of segment area pixels by the area of each pixel in square centimeters.

$$\text{Area of each pixel} = 1 / (\text{resolution}_x \times \text{resolution}_y), \quad (20)$$

where resolution_x and resolution_y are the resolutions of the image in the x and y directions,

3.5. Performance Measures. The evaluation of object detection models commonly relies on the widely used metrics of AP and mAP. These performance measures provide insights into the model's ability to accurately predict bounding boxes. The evaluation considers the counts of true positives (TPs), false positives (FPs), true negatives (TNs), and false negatives (FNs). TP and TN detections are considered correct, while FP and FN detections are regarded as errors.

Precision is a measure that calculates the proportion of TP predictions out of all positive predictions. Precision is particularly valuable when we desire a high level of confidence in our detections. It measures the accuracy of positive predictions, showing how many of the predicted positives are actually TPs. Precision can be calculated using the following formula [31]:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (21)$$

Recall is "sensitivity" or "TP rate." It represents the ratio of correctly predicted positive samples to the total actual positive samples, including both TPs and FNs. Sensitivity assesses the model's capability to accurately identify all positive instances.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (22)$$

mAP serves as a widely employed metric to evaluate object detection models, including Fast R-CNN, YOLO, and Mask R-CNN. It calculates the average of the AP values obtained for each individual class. By computing the AP for each class and subsequently averaging them, mAP provides a comprehensive assessment of the model's performance. This metric takes into consideration both FPs and FNs and effectively captures the balance between precision and recall.

$$\text{mAP} = \frac{1}{n} \sum_{k=1}^{k=n} \text{AP}_k. \quad (23)$$

Here, k denotes the AP of class k , while n represents the total number of classes.

Intersection over Union (IoU) is a measure used in computer vision and image processing to evaluate the accuracy of object detection or segmentation algorithms. It is also known as the Jaccard Index.

$$\text{IoU} = \frac{\text{Area of intersection}}{\text{Area of union}}, \quad (24)$$

where the area of intersection is the area where the predicted object and the ground truth object overlap, and the area of union is the total area encompassed by both the predicted object and the ground truth object.

4. Results and Discussions

The implementation of the proposed model was carried out using Google Colab [32], a Python environment hosted on the Google Cloud platform. The model development was facilitated by PyTorch [33], a widely adopted open-source machine-learning framework for deep learning tasks.

4.1. Parameter Settings. The performance of the model is influenced by the configuration of parameter values. Throughout the training phase, the model was trained using a backpropagation algorithm employing the YOLO loss function, with SGD serving as the optimizer. The training was conducted in several phases. The learning rates were selected through a systematic optimization process to optimize the model's performance. Initially, a lower learning rate of 0.00001 and weight decay of 0.00005 were employed for the first 50 epochs. This conservative learning rate helps the model converge gradually and avoid overshooting the optimal solution, while the smaller weight decay mitigates overfitting during early training. For the subsequent 50 epochs, the learning rate was increased to 0.0001, and weight decay was raised to 0.0005. This decision was based on the understanding that the model would have partially converged by this point, and a higher learning rate could aid in making more significant weight updates for further fine-tuning the model's performance. The increased weight decay provides stronger regularization to prevent overfitting during this phase. Finally, throughout the remaining 100 epochs, the learning rate was maintained at 0.0001, and weight decay remained at 0.0005. The process of adjusting the learning rate is typically guided by monitoring the model's training progress and test performance. The learning rate schedule may be determined through experimentation, starting with a low value and then modifying it based on the model's convergence and test results. This approach ensures a consistent learning rate while allowing the model to continue learning and refining its parameters over an extended period. The performance of the training process is also affected by the batch size and the total number of epochs. In this case, a batch

size of 8 and a momentum of 0.9 were utilized throughout the training process.

4.2. Performance Analysis of the Pothole Detection System. The dataset was divided into training sets, validation sets, and testing sets. The training set includes 2589 images, 740 images, and for testing, 369 images. The model experimented with various combinations of pre-trained networks. Each model is trained for 200 epochs in varied learning rates, achieving a training mAP of 99%, test mAP of 98% for the pothole detector with Xception CBAM. The model is evaluated using various metrics. The precision–recall, and loss graph is obtained for the MakeML dataset, and the prepared dataset is illustrated in Figures 6 and 7. A precision–recall graph is a plot that gives a relationship between precision and recall. Precision, also known as positive predictive value, is the ratio of TP predictions to the sum of TP and FP predictions, while recall, also known as sensitivity or TP rate, is the ratio of TP predictions to the sum of TP and FN predictions.

A precision–recall graph would have precision on the y -axis (vertical axis) and recall on the x -axis (horizontal axis). The graph would show how precision and recall change as the threshold for the model is varied. The curve on the graph would typically start at a high precision and low recall point and gradually move towards a lower precision and higher recall point as the threshold is lowered. The results show that Xception pothole detector curves do not exhibit a balance between precision and recall. Due to overfitting on the training data, the model's capacity to generalize to unfamiliar data may be compromised, leading to inadequate recall performance. The model may have learned to memorize the training data rather than capture the underlying patterns, leading to poor performance on new data. The performance analysis of the Xception pothole detector in the prepared dataset is shown in Figure 6. Table 2 presents the comprehensive efficiency measures of the model utilizing both datasets. The outcomes demonstrate that the model successfully identified the presence of potholes.

The detection results for various models are depicted in Figure 8. In Figure 8, the bounding boxes are displayed in green to represent the actual boxes, while the red color indicates the predicted boxes. The results depicted in Figure 8, which provide a comparative analysis of various experimental models, clearly establish the superiority of Xception with the attention mechanism (Xception CBAM) over VGG16 CBAM and Inception CBAM. The notable alignment between the predicted boxes and target boxes in Xception CBAM strongly indicates its proficiency in accurately localizing objects within images. Several factors contribute to the exceptional performance of Xception CBAM. First, Xception's utilization of depthwise separable convolutions enables it to capture intricate details and fine-grained features within images. When complemented by the channel attention mechanism (CBAM), Xception gains the ability to selectively highlight crucial channels and spatial regions within its feature maps. The attention mechanism significantly enhances the model's capability to focus on pertinent image regions, resulting in markedly improved prediction accuracy.

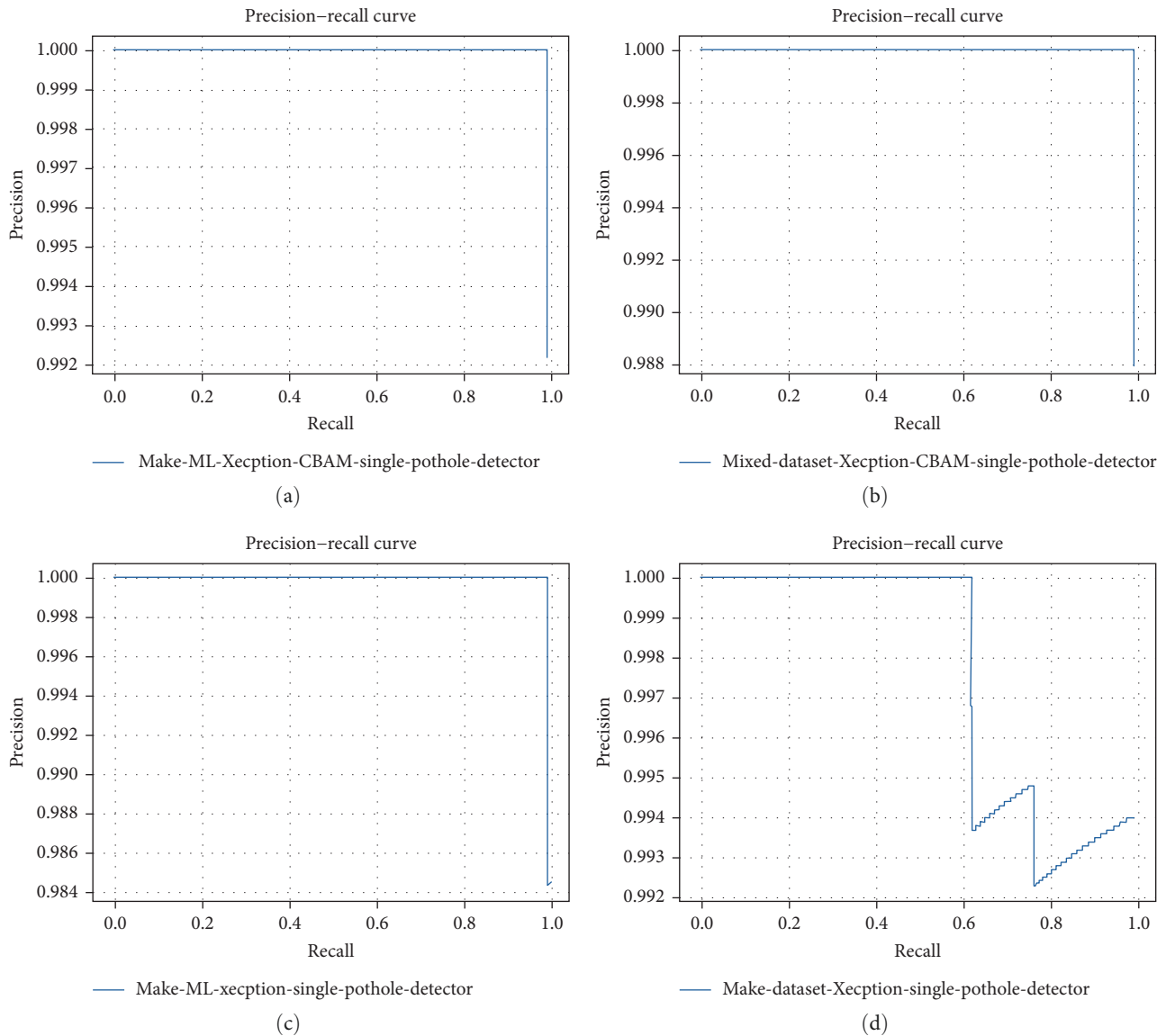


FIGURE 6: Performance analysis using precision-recall graph: (a) MakeML in Xception-CBAM, (b) mixed dataset in Xception-CBAM, (c) MakeML in Xception, and (d) mixed dataset in Xception. CBAM, convolutional block attention module.

In contrast, VGG16 CBAM, while a robust architecture, may be less effective at capturing fine details due to its reliance on conventional convolutional layers and its relatively limited depth compared to Xception. Similarly, inception CBAM, despite offering multiscale feature extraction, may not achieve the same level of proficiency in capturing intricate features as Xception CBAM.

A loss graph, also known as a training loss curve or loss vs. epoch plot, is a graphical representation of the loss value of a model during the training process and validation process. Loss is a measure of the error between the predicted output and the actual output for a given set of input data. The loss graph shows how the loss value changes over the course of training iterations or epochs. Typically, the loss graph has the epoch (or iteration) number on the x -axis (horizontal axis) and the loss value on the y -axis (vertical axis). The loss value is calculated based on the model's predictions and the

actual targets or labels during each training iteration or epoch. The loss graph of various detection models is shown in Figure 7. In a well-trained model, the loss is expected to decrease gradually over time as the model learns to make better predictions. The initial loss value may be relatively high, but it should decrease over epochs as the model updates its parameters to optimize its predictions. The loss graph may exhibit fluctuations or noise due to the stochastic nature of the training process, but the overall trend should be toward lower loss values. Figure 7 illustrates the progression of the loss curve over the course of multiple iterations, demonstrating a clear decrease as the model converges. Additionally, the mAP values for YOLO-V1, YOLO-V3, and the pothole detector on the MakeML pothole dataset are presented in Table 3.

Xception-CBAM surpasses YOLO-V1 and YOLO-V3 due to its ability to selectively focus on relevant regions within an image, enhancing both accuracy and efficiency.

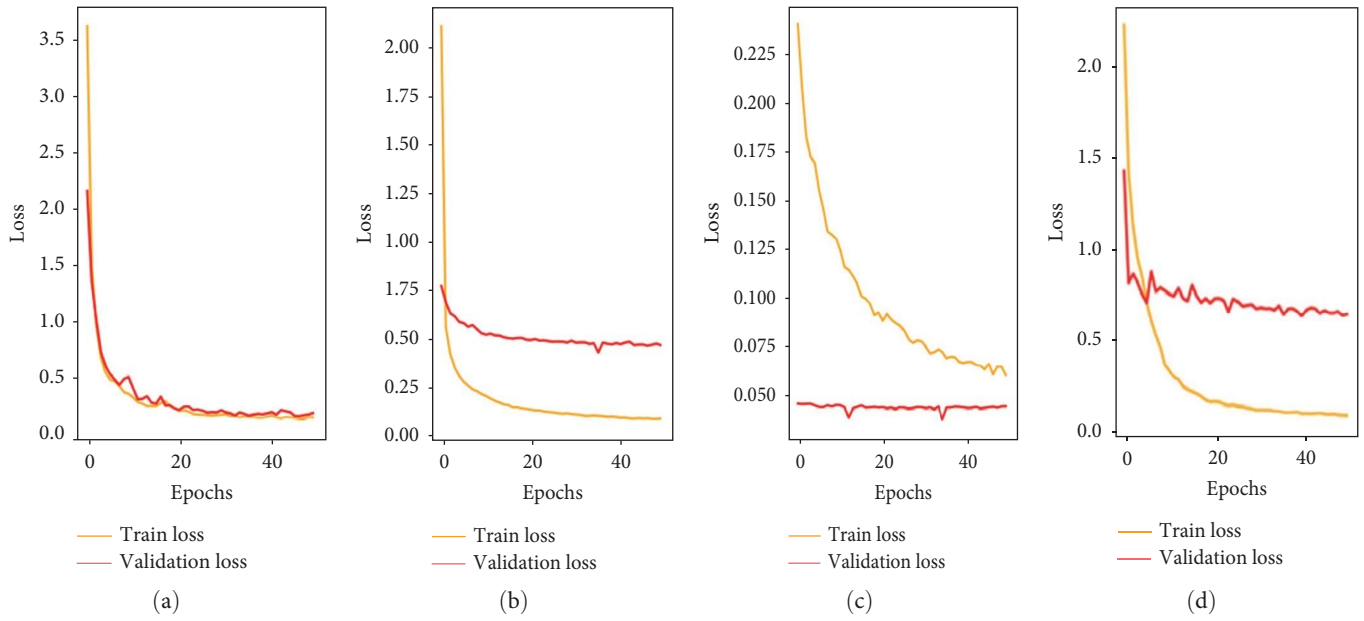


FIGURE 7: Loss curve (a) MakeML in Xception-CBAM, (b) mixed dataset in Xception-CBAM, (c) MakeML in Xception, and (d) mixed dataset in Xception. CBAM, convolutional block attention module.

TABLE 2: Efficiency measures of pothole detector with various backbones [34, 35].

Models	Train mAP	Test mAP	Train AP	Test AP	Precision	IoU
VGG-16 [34]	0.97	0.96	0.97	0.96	0.82	0.93
Inception [35]	0.94	0.65	0.94	0.65	0.99	0.81
Xception	0.93	0.67	0.93	0.67	0.98	0.81
VGG-16-CBAM	0.97	0.37	0.97	0.37	0.26	0.43
Inception-CBAM	0.96	0.79	0.96	0.79	0.97	0.75
Xception-CBAM	0.99	0.97	0.99	0.97	0.99	0.86

Note: Bold values indicate the results of the top-performed model.

Unlike YOLO-V1, which uses a fixed grid and lacks attention, and YOLOV3, which utilizes a complex multiscale approach, YOLO, with attention dynamically allocates attention to significant object regions. The attention mechanism allows the model to concentrate on critical details, improving object localization and recognition. Consequently, YOLO with attention achieves better performance in object detection tasks while maintaining computational efficiency, making it a superior choice for real-time applications and scenarios with complex backgrounds and multiple objects.

4.3. Comparison with State-of-the-Art Methods. The works in the literature mainly dealt with the detection of potholes. We conducted a comparative analysis of our Xception-CBAM-Pothole-Detector with other advanced techniques and found that it outperformed them in terms of detection accuracy. For instance, Asad et al. [14] achieved a mAP of 95% using YOLOv5 on a pothole image dataset (PID). On the other hand [6], utilized a self-collected dataset and obtained an mAP of 45.4% with MVGG-16. In a separate study by researchers in [36], they applied YOLOv5 to the PID and attained an mAP of 74.48%. The main difference, when we experiment is that Xception with attention is more scalable

than YOLOv5 and modified VGG. This is because the attention architecture can be easily scaled up or down by adjusting the number of layers and filters. A comparison of our results with other state-of-the-art techniques is presented in Table 4.

While the accuracy of the model is a critical factor, computational complexity and efficiency are equally important, especially for real-time applications such as onboard vehicle systems. The proposed Xception-CBAM model strikes an effective balance between high accuracy and efficient processing. In our experiments, the Xception-CBAM model achieved an inference time of 0.028 s per image, which, while slightly higher than models like YOLOv5 (0.007 s per image), remains competitive. This demonstrates that the Xception-CBAM model not only delivers superior accuracy but also maintains a reasonable processing speed, making it well-suited for real-time applications.

5. Discussions

In this work, we introduce a new method to enhance pothole detection accuracy by incorporating an attention mechanism into the backbone network and estimating the pothole area. Our approach targets regions where potholes are most likely

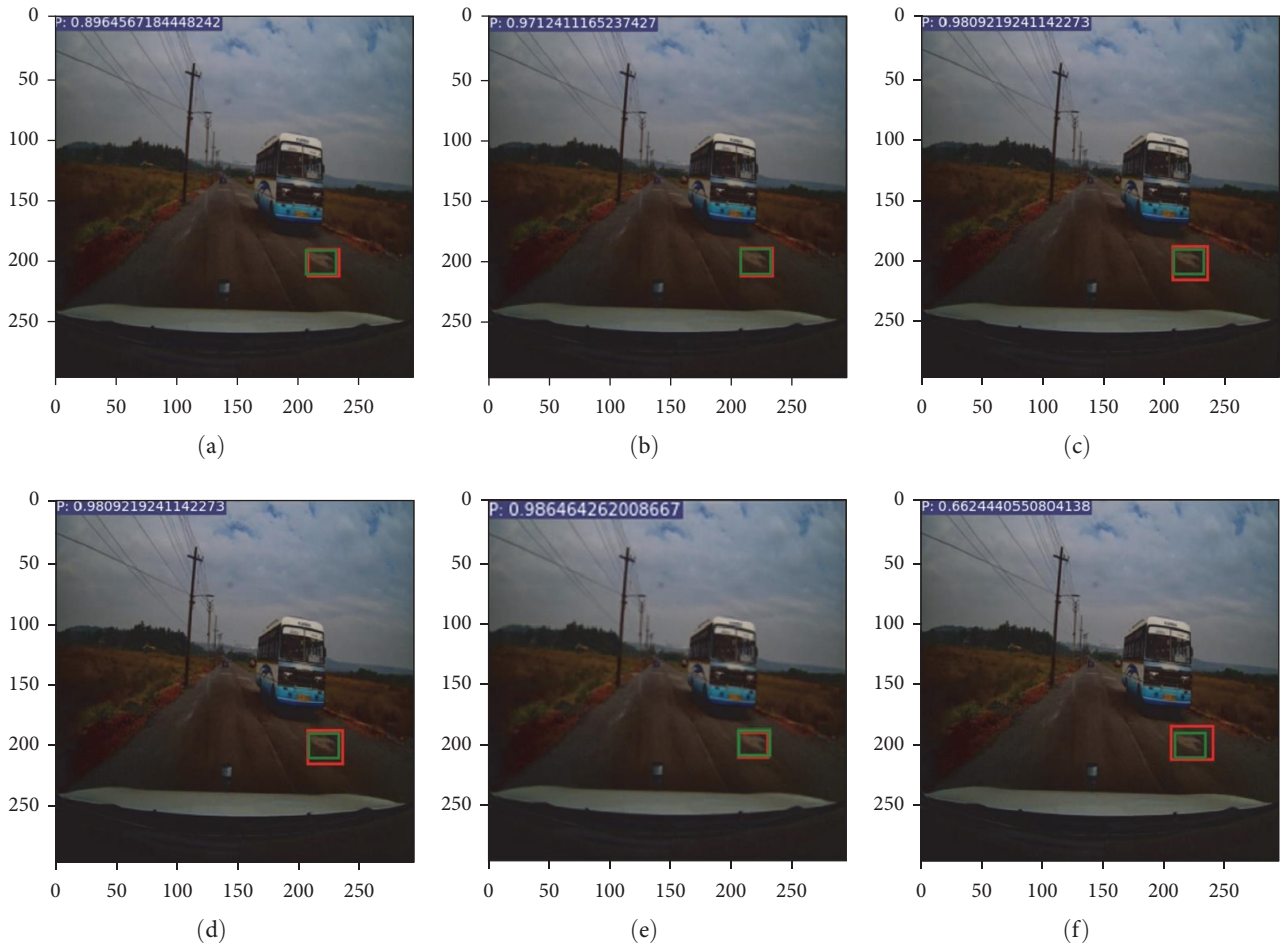


FIGURE 8: Detection results of various models in the mixed dataset: (a) Xception, (b) Xception-CBAM, (c) inception-V3, (d) inception-V3-CBAM, (e) VGG-16, and (f) VGG-16-CBAM. CBAM, convolutional block attention module.

TABLE 3: Comparison with YOLO algorithm.

Dataset	Model	mAP (%)
MakeML pothole	YOLO-V1	54
	YOLO-V3 [36]	64
	Proposed model	99

Note: Bold values indicate the results of the top-performed model.

TABLE 4: Comparison with state-of-the-art methods.

Work	Model	mAP (%)
Asad et al. [14]	YOLO-V5	95
Ahmed et al. [6]	MVGG-16	45.4
Park et al. [36]	YOLO-V5	74.48
Proposed model	Pothole detector with Xception-CBAM	99.21

Note: Bold values indicate the results of our proposed model that gave the top experimental results.

to occur, thereby boosting the overall performance of the system. We modified the YOLO algorithm by tweaking the model architecture through the removal and addition of layers, resulting in improved mAP scores. The pothole detector leverages the

Xception backbone and integrates attention mechanisms to address challenges in detecting small or densely clustered objects. Xception's depthwise separable convolutions enhance feature extraction and improve the detection of small, irregular potholes, outperforming the standard YOLO algorithm. This backbone's ability to generalize across various tasks is advantageous for detecting small objects with diverse appearances and contexts. Additionally, the reduction in parameters from depthwise separable convolutions helps mitigate overfitting compared to the YOLO model. We utilized the YOLO model for experiments due to its low parameter count, which facilitates the area estimation process. The inclusion of attention mechanisms is essential for extracting valuable features, focusing specifically on pothole regions. To further refine the model's performance and enhance its predictive abilities, we experimented with various parameters such as weight decay and learning rate. To ensure robustness, the model was trained on a diverse dataset with varying lighting conditions and different road surfaces. During training, we tested different pretrained models and found that Xception-CBAM performed best in terms of accurate predictions. The proposed model marks a significant advancement in computer vision, offering the potential to significantly improve the precision and reliability of pothole detection and area estimation in real-world scenarios. Future studies could

integrate depth measurement using a depth map to continuously gauge the depth of potholes.

6. Conclusion

This work presents an efficient pothole detector for detecting potholes in roads to facilitate maintenance work. The experiments conducted in this work utilize a carefully prepared dataset that includes diverse pothole images captured under varying daylight conditions, various road conditions, and with different shapes and sizes of potholes. The pothole dataset is trained using different backbones in the pothole detector, utilizing the YOLO algorithm, including Xception, Inception, VGG-16, Xception-CBAM, Inception-CBAM, and VGG-16-CBAM. The experimental findings demonstrate that the pothole detector with Xception-CBAM backbone achieves the highest mAP of 99% on both a mixed dataset and the MakeML pothole dataset, surpassing other YOLO models. Additionally, computer-based techniques such as grayscale conversion, inverted binary thresholding, median blur, and morphological processes are employed for estimating the area of the potholes. By integrating our method with other computer-aided software, we achieve precise detection and accurate area estimation without encountering issues of box collisions.

7. Future Work

While the current system demonstrates high accuracy in pothole detection, there are several avenues for future research to further enhance its capabilities. One promising direction is the integration of depth measurement using depth maps or stereo vision, which could provide additional spatial information to improve the localization and area estimation of potholes, especially in more complex or cluttered environments. Additionally, exploring the use of more advanced deep learning architectures, such as transformer-based models or attention-enhanced variants of YOLO, could further boost detection accuracy and robustness. Optimizing the model for real-time deployment on edge devices, such as drones or mobile systems, would also be a valuable extension, making the system more accessible for on-the-go road inspections. Furthermore, incorporating a wider variety of road conditions and pothole types from diverse geographical regions could improve the generalization of the model, making it more adaptable to different environments.

Another avenue for future research could involve the incorporation of multimodal data, such as combining visual inputs with sensor data (e.g., accelerometer or GPS data), to create a more comprehensive pothole detection system that operates effectively in a variety of real-world scenarios. By addressing these challenges and exploring these avenues, future work can continue to push the boundaries of pothole detection, contributing to more efficient and proactive road maintenance practices.

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

Consent

The authors affirm that there were no informed consents during the preparation of this manuscript.

Disclosure

This article does not include any studies involving human or animal subjects conducted by the authors.

Conflicts of Interest

The authors declare no conflicts of interest.

Author Contributions

Rufus Rubin conducted data collection and experimented the work. Chinnu Jacob analyzed and validated the experimental results. Sumod Sundar investigated the work and provided critical suggestions. Gabriel Stoian carried out the data curation in detail. Daniela Danculescu performed the formal analysis of the entire research findings. Jude Hemanth supervised the work. All authors were involved in the study's conception and design and equally contributed toward the manuscript writing.

Funding

No funding was received for this research.

Acknowledgments

We would like to sincerely thank Acutro Technologies Pvt Ltd, Trivandrum, for their valuable contribution in providing the dataset used in this study. Their support and generosity were crucial in ensuring the successful completion of our research project.

References

- [1] K. V. Mhetre and A. D. Thube, "Statistical Analysis of Crash Data and Impact of COVID-19 on Road Crashes in Maharashtra, India," *Journal of Engineering and Applied Science* 70, no. 1 (2023): 1–20.
- [2] S. Gupta, P. Sharma, D. Sharma, V. Gupta, and N. Sambyal, "Detection and Localization of Potholes in Thermal Images Using Deep Neural Networks," *Multimedia Tools and Applications* 79, no. 35–36 (2020): 26265–26284.
- [3] N. Ma, J. Fan, W. Wang, et al., "Computer Vision for Road Imaging and Pothole Detection: A State-of-the-Art Review of Systems and Algorithms," *Transportation Safety and Environment* 4, no. 4 (2022): tdac026.
- [4] L. Luo, "Autonomous Pothole Detection Using Deep Region-Based Convolutional Neural Network With Cloud Computing," *Smart Structures and Systems, An International Journal* 24, no. 6 (2019): 745–757.
- [5] G. Ochoa-Ruiz, A. A. Angulo-Murillo, A. Ochoa-Zezzatti, L. M. Aguilar-Lobo, J. A. Vega-Fernández, and S. Natraj, "An Asphalt Damage Dataset and Detection System Based on Retinanet for Road Conditions Assessment," *Applied Sciences* 10, no. 11 (2020): 3974.
- [6] K. R. Ahmed, "Smart Pothole Detection Using Deep Learning Based on Dilated Convolution," *Sensors* 21, no. 24 (2021): 8406.

- [7] C. Jacob and G. C. Menon, "Pathological Categorization of Lung Carcinoma From Multimodality Images Using Convolutional Neural Networks," *International Journal of Imaging Systems and Technology* 32, no. 5 (2022): 1681–1695.
- [8] S. Sundar and S. Sumathy, "An Effective Deep Learning Model for Grading Abnormalities in Retinal Fundus Images Using Variational Auto-Encoders," *International Journal of Imaging Systems and Technology* 33, no. 1 (2023): 92–107.
- [9] S. Sundar and S. Sumathy, "Transfer Learning Approach in Deep Neural Networks for Uterine Fibroid Detection," *International Journal of Computational Science and Engineering* 25, no. 1 (2022): 52–63.
- [10] S. Li, H. Yang, R. Cheng, and H. Ge, "Hybrid Deep Learning Models for Short-Term Demand Forecasting of Online Car-Hailing Considering Multiple Factors," *Transportation Letters* 16, no. 3 (2023): 218–233.
- [11] P. Kumar, K. Hariharan, and M. S. Manikandan, "Hybrid Long Short-Term Memory Deep Learning Model and Dijkstra's Algorithm for Fastest Travel Route Recommendation Considering Eco-Routing Factors," *Transportation Letters* 15, no. 8 (2023): 926–940.
- [12] K. Kumar, M. Kumar, and P. Das, "Traffic Congestion Forecasting Using Multilayered Deep Neural Network," *Transportation Letters* 16, no. 6 (2024): 516–526.
- [13] W. Liu, D. Anguelov, D. Erhan, et al., "SSD: Single Shot Multibox Detector," in *Computer Vision-ECCV 2016: 14th European Conference*, (Springer International Publishing, Amsterdam, The Netherlands, 2016): 21–37.
- [14] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, "Pothole Detection Using Deep Learning: A Real-Time and AI-on-the-Edge Perspective," *Advances in Civil Engineering* 2022 (2022): 9221211, 13.
- [15] M. P. B. and S. K.C., "Enhanced Pothole Detection System Using YOLOX Algorithm," *Autonomous Intelligent Systems* 2, no. 1 (2022): 22.
- [16] J. Li, X. Liang, Y. Wei, T. Xu, J. Feng, and S. Yan, "Perceptual Generative Adversarial Networks for Small Object Detection," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, (Honolulu, HI, USA: IEEE, 2017), 1222–1230.
- [17] J. Zhong, Y. Ma, M. Zhang, R. Xiao, G. Cheng, and B. Huang, "A Pavement Crack Translator for Data Augmentation and Pixel-Level Detection Based on Weakly Supervised Learning," *IEEE Transactions on Intelligent Transportation Systems* 25, no. 10 (2024): 13350–13363.
- [18] O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin, and J. Matas, "DeblurGAN: Blind Motion Deblurring Using Conditional Adversarial Networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (IEEE, 2018): 8183–8192.
- [19] C. Ledig, L. Theis, F. Huszar, J. Caballero, A. Cunningham, and A. Acosta, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition (CVPR)*, (Honolulu, HI, USA: IEEE, 2017), 4681–4690.
- [20] MakeML, "MakeML: Potholes Dataset, Available Online:" <https://makeml.app/datasets/potholes>.
- [21] A. R. Chitholian, "Roboflow: Potholes Dataset," <https://public.roboflow.com/object-detection/pothole>.
- [22] Kaggle Potholes Dataset, <https://www.kaggle.com/datasets/trolololo888/potholes-and-road-damage-with-annotations>.
- [23] Q. Xie, Z. Dai, E. Hovy, T. Luong, and Q. Le, "Unsupervised Data Augmentation for Consistency Training," *Advances in Neural Information Processing Systems* 33 (2020): 6256–6268.
- [24] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification With Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems* 60, no. 6 (2012): 84–90.
- [25] F. Chollet, "Xception: Deep Learning With Depthwise Separable Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, (Honolulu, HI, USA: IEEE, 2017), 1251–1258.
- [26] S. Woo, J. Park, J. Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, eds. V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, (Springer, Cham, 2018): 3–19.
- [27] H. Zhang, H. Zhang, C. Wang, and J. Xie, "Co-Occurrent Features in Semantic Segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (IEEE, 2019): 548–557.
- [28] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, (Las Vegas, NV, USA: IEEE, 2016), 779–788.
- [29] S. Ruder, "An Overview of Gradient Descent Optimization Algorithms," arXiv preprint arXiv: 1609.04747 (2016).
- [30] N. S. Keskar and R. Socher, "Improving Generalization Performance by Switching From Adam to Sgd," arXiv preprint arXiv: 1712.07628 (2017).
- [31] R. Rubin, C. Jacob, S. M. Anzar, and A. Panthakkan, "Mask R-CNN With Multi-Backbones-A Comparative Analysis," in *2022 5th International Conference on Signal Processing and Information Security (ICSPIS)*, (Dubai, United Arab Emirates: IEEE, 2022), 55–60.
- [32] E. Bisong and Google colab, *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners* (Apress Berkeley, CA, 2019): 59–64.
- [33] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems* 32 (2019).
- [34] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Largescale Image Recognition," arXiv preprint arXiv: 1409.1556 (2014).
- [35] C. Szegedy, W. Liu, Y. Jia, et al., "Going Deeper With Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, (IEEE, 2015): 1–9.
- [36] S.-S. Park, V. T. Tran, and D. E. Lee, "Application of Various YOLO Models for Computer Vision-Based Real-Time Pothole Detection," *Applied Sciences* 11, no. 23 (2021): 11229.